

Jenkins on Kubernetes

Helm Installation Guide

Environment: kubeadm | Rocky Linux 9.7 | Kubernetes v1.28.15 | containerd | 1 control-plane + 2 workers

Purpose: Install a second Jenkins controller with the official Jenkins Helm chart, without changing the existing manual Jenkins installation.

Topology: master 192.168.178.49; node01 192.168.178.51 (existing Jenkins); node02 192.168.178.52 (Helm Jenkins, if suitable).

1. Safety and deployment design

Your manual Jenkins uses namespace jenkins, a local PV on node01, and NodePort 30080. For this exercise, Helm uses namespace jenkins-helm, its own persistent volume on node02, and a different NodePort 30081. Do not reuse the manual Jenkins PVC or its data directory while both controllers run.

Storage: This guide creates a statically provisioned local PV, not a dynamic provisioner. Local PVs are tied to a node and are not highly available. A 10Gi PV capacity is metadata, not a disk quota. Confirm node02 has enough free space and memory before proceeding.

Compatibility: Kubernetes v1.28 is out of upstream support. Newer Jenkins chart releases may require newer Kubernetes versions. Inspect the chart requirements and run `helm template` before installation; if incompatible, upgrade the cluster or use a compatible chart release. Do not blindly install the latest chart.

2. Prerequisites

```
kubectl get nodes -o wide
kubectl get pods -n kube-system
kubectl get pv,pvc -A
helm version
kubectl top nodes # optional; requires metrics-server
```

Helm must be installed on the machine where you run these commands. Example on macOS: `brew install helm`. On Linux, use the official Helm installation instructions and verify the binary. Ensure your `kubeconfig` points to the home-lab cluster.

3. Prepare dedicated local storage on node02

Run the following on node02 (not on master). The Jenkins image generally runs as UID/GID 1000. Check filesystem permissions and SELinux context if writing fails.

```
ssh root@192.168.178.52
mkdir -p /data/jenkins-helm
chown 1000:1000 /data/jenkins-helm
chmod 750 /data/jenkins-helm
df -h /data/jenkins-helm
getenforce
exit
```

SELinux Permissive permits access despite policy denials, but standard Unix ownership and permissions still apply. Avoid disabling SELinux as a troubleshooting shortcut.

4. Create namespace and static PersistentVolume

```
kubectl create namespace jenkins-helm
mkdir -p ~/jenkins-helm && cd ~/jenkins-helm
```

Save the following as jenkins-helm-pv.yaml:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: jenkins-helm-pv
spec:
  capacity:
    storage: 10Gi
  volumeMode: Filesystem
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: jenkins-helm-local
  local:
    path: /data/jenkins-helm
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values: [node02]

kubectl apply -f jenkins-helm-pv.yaml
kubectl get pv jenkins-helm-pv
```

5. Add the official Jenkins Helm chart

```
helm repo add jenkins https://charts.jenkins.io
helm repo update
helm search repo jenkins/jenkins --versions | head -20
helm show chart jenkins/jenkins
helm show values jenkins/jenkins > chart-default-values.yaml
```

Check kubeVersion (if declared), chart/app version, and available controller/persistence/service settings. Helm chart values change between versions. Choose a chart version compatible with your cluster, and set it explicitly in the commands below.

6. Create the Helm values file

The official Jenkins chart supports a pre-existing claim. We will create a PVC explicitly to bind the local PV, then tell the chart to use it. This keeps the storage setup visible for learning.

Save as jenkins-helm-pvc.yaml:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: jenkins-helm-pvc
  namespace: jenkins-helm
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: jenkins-helm-local
  volumeName: jenkins-helm-pv
  resources:
    requests:
      storage: 10Gi

kubectl apply -f jenkins-helm-pvc.yaml
```

```
kubectl get pvc -n jenkins-helm
```

Save as values.yaml. Check the exact keys against chart-default-values.yaml for your selected chart version:

```
controller:
  serviceType: NodePort
  nodePort: 30081
  servicePort: 8080
  targetPort: 8080
  resources:
    requests:
      cpu: "500m"
      memory: "1Gi"
    limits:
      cpu: "2"
      memory: "3Gi"

persistence:
  enabled: true
  existingClaim: jenkins-helm-pvc

agent:
  enabled: true
```

Avoid putting an administrator password in a Git-tracked values.yaml. The chart can manage credentials via a Kubernetes Secret; consult the selected chart defaults for adminSecret/adminPassword configuration. The example uses chart defaults for initial credentials.

7. Render and inspect the manifests before installation

Replace CHART_VERSION with the version you selected in Step 5. For a versioned installation, use the same value for template, install, and upgrade.

```
helm template jenkins jenkins/jenkins \
  --namespace jenkins-helm \
  --version CHART_VERSION \
  -f values.yaml > rendered-jenkins.yaml

# Inspect key objects and settings
grep -nE "kind:|nodePort:|claimName:|serviceAccountName:" \
  rendered-jenkins.yaml | head -90
```

Confirm the generated Service is NodePort 30081, the workload mounts jenkins-helm-pvc, and the chart does not create an unwanted second PVC. Confirm there is no chart incompatibility with Kubernetes v1.28.

8. Install Jenkins with Helm

```
helm install jenkins jenkins/jenkins \
  --namespace jenkins-helm \
  --version CHART_VERSION \
  -f values.yaml \
  --wait --timeout 15m
```

The release name is jenkins, but it is separate from the manual installation because it lives in jenkins-helm. The first startup can take several minutes while Jenkins initializes and plugins download.

9. Verify Kubernetes and Helm resources

```
helm list -n jenkins-helm
helm status jenkins -n jenkins-helm
kubectl get pods -n jenkins-helm -o wide
kubectl get svc -n jenkins-helm
kubectl get pvc -n jenkins-helm
kubectl get pv jenkins-helm-pv
kubectl get events -n jenkins-helm --sort-by=.lastTimestamp
```

Expected: controller Pod Ready, scheduled on node02 due to the local PV, PVC Bound, and Service exposed on NodePort 30081. The chart may create multiple Services and supporting resources, including agent connectivity and service accounts.

10. Open Jenkins and retrieve the admin password

From your Mac on the home LAN, try `http://192.168.178.52:30081`. If needed, use port-forward instead of NodePort:

```
kubectll -n jenkins-helm port-forward svc/jenkins 8081:8080
# Open http://localhost:8081
```

For many official chart versions, the admin credentials are stored in a release-named Secret. Verify the actual Secret name with `kubectl get secrets -n jenkins-helm`. A common retrieval command is:

```
kubectll get secret jenkins -n jenkins-helm \
-o jsonpath="{.data.jenkins-admin-password}" | base64 -d
echo
```

If the Secret name or key differs, inspect the chart NOTES and Secret names. The default administrator username is commonly admin. Do not print credentials into shared logs or commit them to Git.

11. Configure Jenkins after first login

- Confirm the Jenkins URL matches how you access it (NodePort or port-forward).
- Check plugin status and update center connectivity. A forward HTTP proxy is only needed if your network requires one for outbound traffic.
- For Kubernetes agents, configure the Kubernetes cloud and service account permissions. Start with a simple pipeline that prints environment information; then add image builds using a suitable in-Pod builder.
- Keep the controller private to the LAN; do not expose this unauthenticated HTTP endpoint to the internet. Add Ingress + HTTPS later if desired.

12. Helm management commands

Inspect current configuration

```
helm get values jenkins -n jenkins-helm
helm get manifest jenkins -n jenkins-helm
helm history jenkins -n jenkins-helm
```

Upgrade configuration

```
helm upgrade jenkins jenkins/jenkins \
-n jenkins-helm --version CHART_VERSION \
-f values.yaml --wait --timeout 15m
```

Rollback to an earlier revision

```
helm history jenkins -n jenkins-helm
helm rollback jenkins REVISION -n jenkins-helm --wait
```

Uninstall the Helm release

```
helm uninstall jenkins -n jenkins-helm
# PVC/PV and node-local data are intentionally retained;
# verify before any manual deletion.
```

Because the PV reclaim policy is Retain, deleting the release does not automatically erase the data. Do not delete the PV/PVC or /data/jenkins-helm until you have backed up Jenkins and confirmed you no longer need the installation.

13. Troubleshooting quick reference

Symptom	Check
PVC Pending	kubectl describe pvc -n jenkins-helm; check PV name, class, capacity
Pod Pending	kubectl describe pod -n jenkins-helm; node02 resources, affinity, taints
CrashLoopBackOff	kubectl logs -n jenkins-helm POD -c jenkins --previous
Permission denied	UID/GID 1000, /data/jenkins-helm ownership, SELinux audit logs
UI unreachable	kubectl get svc -n jenkins-helm; firewall; test port-forward
Plugin download fails	Pod DNS, outbound HTTPS, proxy requirements, certificates
Helm template/install fails	Check chart version, kubeVersion, values keys, Kubernetes compatibility

14. Manual vs Helm: what changed?

Manual: You write PV, PVC, ServiceAccount, RBAC, StatefulSet and Services yourself and manage changes with kubectl apply. Helm: You still define storage explicitly for this exercise, but the chart generates and tracks the Jenkins workload, Services, RBAC, configuration, and upgrades as a release. Helm is a packaging and release-management layer over Kubernetes manifests.

References

Official Jenkins Helm chart: <https://github.com/jenkinsci/helm-charts>

Jenkins Kubernetes plugin: <https://plugins.jenkins.io/kubernetes/>

Kubernetes local PV: <https://kubernetes.io/docs/concepts/storage/volumes/#local>

Helm documentation: <https://helm.sh/docs/>